

Question 3: Canny Edge Detection Implementation

Pushpendra Uikey

September 12, 2026

Overview

This section details the implementation of a Canny edge detector from scratch. The algorithm was tested on three images: ‘butterfly.png’, ‘paithaniEdge.png’, and ‘rangoli.png’. The primary objective is to reliably extract the major structural boundaries of objects within the scene while minimizing false positives from internal textures or noise.

Methodology

The Canny edge detector is a multi-stage pipeline designed to find the optimal edges in an image. My implementation breaks this down into four distinct phases:

1. **Gaussian Smoothing:** The raw image is convolved with a 2D Gaussian filter. This step is critical; without it, the derivative filters in the next stage would amplify high-frequency noise, resulting in a chaotic edge map.
2. **Gradient Computation:** 3×3 Sobel operators are used to compute the horizontal (G_x) and vertical (G_y) derivatives. These are combined to calculate the gradient magnitude ($\sqrt{G_x^2 + G_y^2}$) and the gradient direction ($\arctan(G_y/G_x)$). The directions are normalized to a 0-180 degree scale.
3. **Non-Maximum Suppression (NMS):** This stage thins the thick gradient responses down to a single pixel width. For every pixel, the algorithm checks its neighbors along the line of the gradient direction. If the central pixel is not the local maximum, its value is suppressed to zero.
4. **Hysteresis Thresholding:** Rather than using a single hard cutoff, two thresholds are used: T_{high} and T_{low} . Any pixel above T_{high} is instantly classified as a "strong" edge. Pixels falling between the two thresholds are classified as "weak." A Depth-First Search (DFS) algorithm then tracks along the strong edges, upgrading any connected weak pixels into strong edges. This prevents continuous lines from breaking apart due to slight localized drops in contrast.

Results and Parameter Tuning

Because the physical structure and contrast vary wildly between the three test images, the parameters (σ , T_{low} , T_{high}) were independently tuned to yield the cleanest possible edge maps.

Butterfly

The butterfly image contains sharp external boundaries but is filled with intricate, high-frequency patterns on the wings. A baseline smoothing ($\sigma = 1.0$) paired with standard thresholds ($T_{low} = 0.05, T_{high} = 0.15$) struck the right balance, capturing the delicate wing veins and antennae without the image dissolving into noise.

butterfly.png | sigma=1.0, t_low=0.05, t_high=0.15



Figure 1: Input image alongside the binary edge map for 'butterfly.png'.

Paithani Textile

Textiles inherently possess a massive amount of micro-texture due to the woven threads. To prevent the edge detector from simply outlining every single thread, heavy smoothing ($\sigma = 2.0$) and aggressive thresholds ($T_{low} = 0.08, T_{high} = 0.25$) were mandatory. This forced the algorithm to ignore the fabric's weave and successfully isolate the major geometric boundaries of the peacocks and floral motifs.

paithaniEdge.png | sigma=2.0, t_low=0.08, t_high=0.25

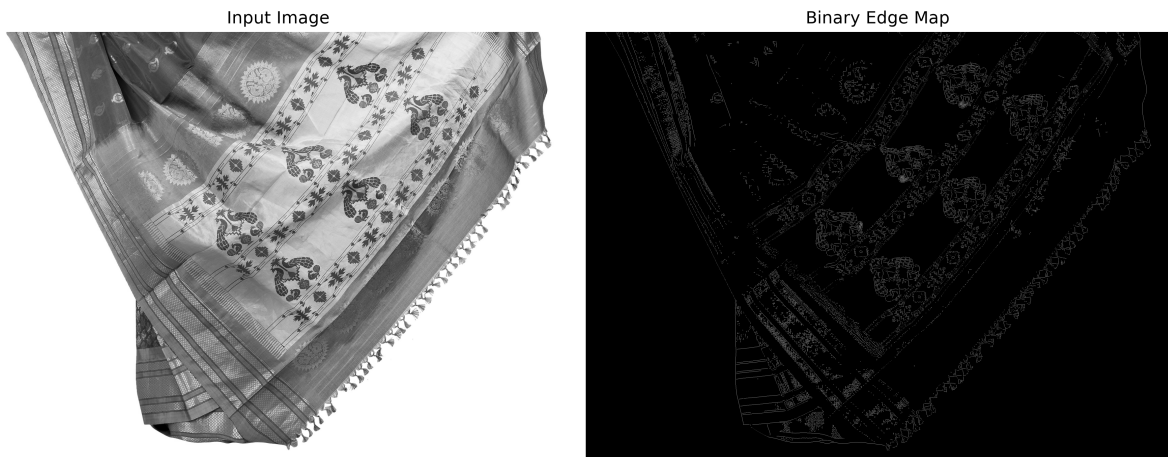


Figure 2: Input image alongside the binary edge map for 'paithaniEdge.png'.

Rangoli

The Rangoli image features high-contrast white powder against a dark floor, but also includes intense, blooming light sources that create severe gradient spikes. Baseline parameters ($\sigma = 1.0$, $T_{low} = 0.05$, $T_{high} = 0.15$) performed perfectly here, extracting the crisp geometric rings and floral patterns while successfully navigating the harsh lighting blowouts.

rangoli.png | sigma=1.0, t_low=0.05, t_high=0.15

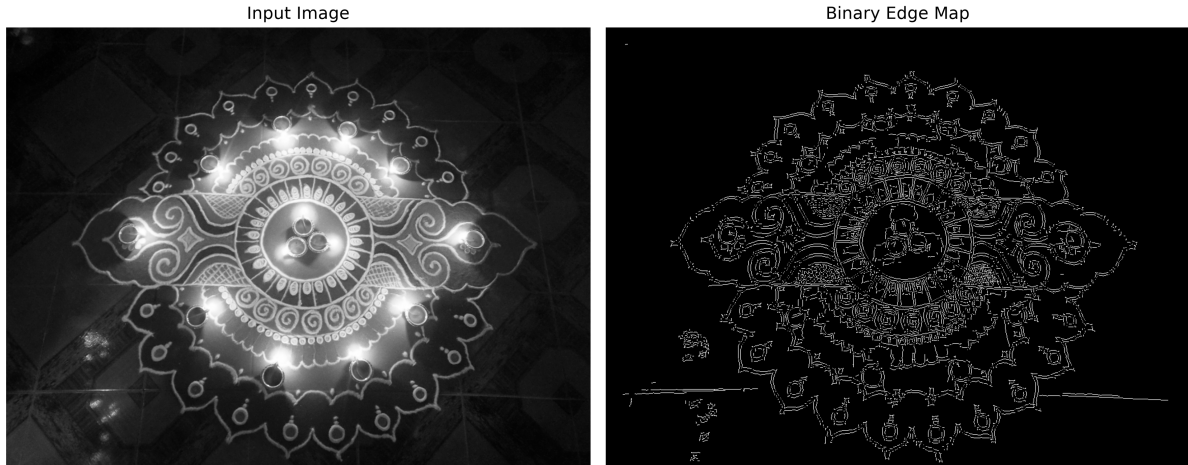


Figure 3: Input image alongside the binary edge map for ‘rangoli.png’.